

Funcionamiento del sistema

Arquitectura y flujo completo — Sistema de Reconocimiento Facial y Tracking por Zonas

Proyecto: reconocimiento-facial Documento 1 de 2

ÍNDICE

1. Componentes principales
2. Piezas del backend
3. Flujo completo: de la cámara al reporte
4. Modelo de datos
5. Reconocimiento facial: cómo decide quién es quién
6. Empleados activos e inactivos
7. Cámaras en vivo vs. tracking
8. Despliegue

Este documento explica la arquitectura y el flujo completo del software, de punta a punta: desde que una cámara capta un rostro hasta que queda disponible el reporte de recorrido de un empleado.

1. Componentes principales

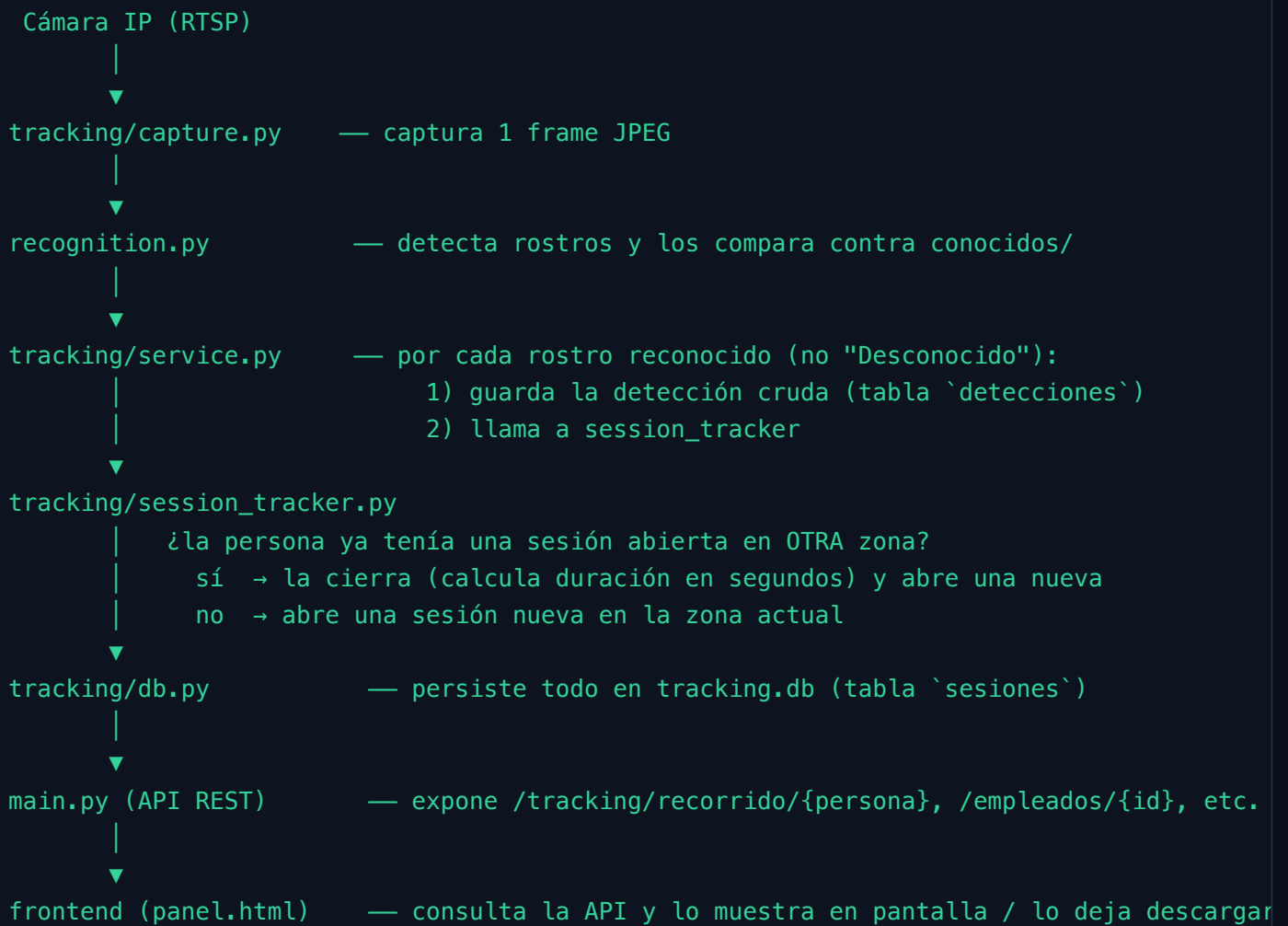
```
reconocimiento-facial/  
├─ backend/      API en Python (FastAPI) + reconocimiento facial + tracking  
└─ frontend/     Interfaz web estática (HTML/CSS/JS, sin frameworks)
```

El **backend** es el único componente con lógica real: recibe imágenes, las procesa, guarda resultados en una base de datos SQLite y expone todo por una API REST. El **frontend** es solo una interfaz que consume esa API — no hace ningún procesamiento de imágenes por su cuenta.

2. Piezas del backend

Archivo / carpeta	Responsabilidad
<code>main.py</code>	Define todos los endpoints de la API (FastAPI)
<code>recognition.py</code>	Carga las fotos de <code>conocidos/</code> , genera sus codificaciones faciales y compara rostros nuevos contra ellas
<code>empleados.py</code>	Alta/edición/baja de empleados, y gestión de sus fotos de referencia
<code>camaras.py</code>	Alta/baja de cámaras, prueba de conexión, generación del stream en vivo (MJPEG)
<code>formato.py</code>	Formato consistente de fechas y duraciones (siempre <code>HH:MM:SS</code> , nunca minutos decimales)
<code>reportes.py</code>	Genera el archivo de texto descargable con el recorrido de un empleado
<code>tracking/capture.py</code>	Arma la URL RTSP y captura un frame de una cámara
<code>tracking/db.py</code>	Todo el acceso a la base de datos SQLite (<code>tracking.db</code>)
<code>tracking/service.py</code>	El loop en background que muestrea las cámaras periódicamente
<code>tracking/session_tracker.py</code>	Decide cuándo abrir/cerrar una "sesión" de permanencia en una zona

3. Flujo completo: de la cámara al reporte



Ciclo de muestreo (tracking/service.py)

El servicio de tracking corre en un hilo en background, separado del resto de la API. Cada `TRACKING_INTERVALO_SEGUNDOS` (configurable en `.env`, por defecto 3 segundos):

1. Pide a la base de datos la lista de cámaras que **ya tienen una zona asignada** (las que no tienen zona quedan excluidas del tracking a propósito).
2. Procesa todas las cámaras **en paralelo** (un hilo por cámara) para no acumular demora — si se hiciera secuencial, el tiempo real entre muestras crecería con la cantidad de cámaras.
3. Por cada cámara: conecta por RTSP, toma un frame, corre el reconocimiento, y si hay una coincidencia (no "Desconocido"), registra la detección y actualiza la sesión.

Lógica de sesiones (permanencia por zona)

Una "sesión" representa un tramo continuo de tiempo que una persona pasó en una zona. El criterio es simple:

- Si la persona **no tenía sesión abierta**, se abre una nueva en la zona donde apareció.
- Si la persona **ya tenía una sesión abierta en la misma zona**, no pasa nada (sigue ahí).

- Si la persona **tenía una sesión abierta en OTRA zona**, esa sesión se cierra (con **salida** = ahora, y **duración** = ahora – entrada) y se abre una sesión nueva en la zona nueva.

NOTA

Esto es lo que permite reconstruir un recorrido tipo: "Depósito 14:02–14:20 (00:18:00), Atención al cliente 14:22–14:35..." sin tener que hacer cálculos al momento de consultar — la duración queda calculada y guardada en el mismo instante en que se detecta el cambio de zona.

4. Modelo de datos (SQLite, **backend/tracking.db**)

Tabla	Para qué sirve
empleados	Perfiles: nombre, apellido, cargo, legajo, estado (activo/inactivo)
zonas	Áreas físicas (ej. "Depósito") creadas desde el panel
camaras	IP, puerto, usuario, contraseña y zona asignada de cada cámara
detecciones	Cada vez que una cámara reconoce a alguien (log crudo, sin agrupar)
sesiones	Los tramos de permanencia ya agrupados por zona (entrada, salida, duración)

5. Reconocimiento facial: cómo decide quién es quién

recognition.py usa la librería **face_recognition** (basada en **dlib**):

1. Al iniciar (o al llamar **/recargar-conocidos/**), recorre **backend/conocidos/** y genera una "codificación" (un vector numérico) por cada foto de referencia.
2. Cuando llega una imagen nueva, detecta los rostros presentes y genera su codificación.
3. Compara esa codificación contra **todas** las codificaciones conocidas y se queda con la de menor distancia.
4. Si esa distancia es menor o igual a **TOLERANCIA** (en **recognition.py**), se considera una coincidencia; si no, se marca como "Desconocido".

Cuantas más fotos de referencia tenga una persona (distintos ángulos, luz, con/sin barba, de día/de noche), más robusto es el reconocimiento — por eso el panel permite cargar varias fotos por empleado, incluyendo capturas tomadas directamente con cámara.

NOTA

La visión nocturna infrarroja (blanco y negro, granulada) da distancias más altas que una foto diurna a color, incluso comparando a la misma persona. Por eso conviene agregar al menos una foto de referencia tomada en las mismas condiciones (de noche, con la misma cámara) para cada persona que se quiera reconocer en ese horario.

6. Empleados activos e inactivos

Desactivar un empleado **no borra sus datos ni su historial**: mueve sus fotos de referencia de `conocidos/` a `conocidos/_inactivos/` y recarga el reconocedor. Esto hace que el sistema deje de reconocerlo (no se generan nuevas detecciones/sesiones a su nombre) sin perder el legajo, cargo ni el recorrido ya registrado. Reactivarlo devuelve las fotos a `conocidos/` y vuelve a incluirlo en el reconocimiento.

7. Cámaras en vivo vs. tracking

Son dos conexiones RTSP independientes a la misma cámara:

- **Tracking** (`tracking/service.py`): usa el stream principal (canal 1, calidad completa), conecta, toma un frame, y se desconecta — cada `TRACKING_INTERVALO_SEGUNDOS`.
- **Vista en vivo** (`camaras.py`, endpoint `/camaras/{id}/stream`): usa el sub-stream (más liviano) y mantiene la conexión abierta mientras el usuario tenga esa cámara abierta en el panel, entregando frames como MJPEG.

Cámaras económicas suelen soportar pocas conexiones RTSP simultáneas — por eso la vista en vivo se activa cámara por cámara (no todas a la vez por defecto) y no comparte conexión con el tracking.

8. Despliegue

El backend está pensado para correr en un VPS vía Docker (`Dockerfile` + `docker-compose.yml` en `backend/`). El frontend son archivos estáticos que se pueden servir desde cualquier servidor HTTP (o el mismo VPS). Ver *Manual de uso* para los pasos concretos.